Chapter 1

Functions of Vectors

Now that we have functions and vectors, we'll put them together to write functions that take vectors as input variables and return vectors as output variables. You'll also see two patterns for computing with vectors: existential and universal quantification.

1.1 Functions and Vectors

In this section we'll look at common patterns involving functions and vectors, and you will learn how to write a single function that can work with vectors as well as scalars.

1.1.1 Vectors as Input Variables

Since many of the built-in functions take vectors as arguments, it should come as no surprise that you can write functions that take vectors as input. Here's a simple (but not very useful) example:

```
function res = display_vector(X)
    for i=1:length(X)
        display(X(i))
    end
end
```

There's nothing special about this function. The only difference from the scalar functions we've seen is that this one uses a capital letter to remind us that `X` is a vector.

Using `display_vector` doesn't actually return a value; it just displays the elements of the vector it gets as an input variable:

```
>> display_vector(1:3)
    1
    2
    3
```

Here's a more interesting example that encapsulates the code from Listing 4.2 on page 45 to add up the elements of a vector:

```
function res = mysum(X)
    total = 0;
    for i=1:length(X)
        total = total + X(i);
    end
    res = total;
end
```

I called this function `mysum` to avoid a collision with the built-in function `sum`, which does pretty much the same thing.

Here's how you call it from the Command Window:

```
>> total = mysum(1:3)
total = 6
```

Because this function has an output variable, I made a point of assigning it to a variable.

1.1.2 Vectors as Output Variables

There's also nothing wrong with assigning a vector to an output variable. Here's an example that encapsulates the code from Listing 4.3 on page 45:

```
function res = mysquare(X)
    for i=1:length(X)
        Y(i) = X(i)^2;
    end
    res = Y;
end
```

This function squares each element of `X` and stores it as an element of `Y`. Then it assigns `Y` to the output variable, `res`. Here's how we use this function:

```
>> V = mysquare(1:3)
V = 1     4     9
```

The input variable is a vector with the elements `1,2,3`. The output variable is a vector with the elements `1,4,9`.

1.1.3 Vectorizing Functions

Functions that work on vectors will almost always work on scalars as well, because MATLAB considers a scalar to be a vector with length 1.

```
>> mysum(17)
ans = 17
```

```
>> mysquare(9)
ans = 81
```

Unfortunately, the converse isn't always true. If you write a function with scalar inputs in mind, it might not work on vectors.

But it might! If the operators and functions you use in the body of your function work on vectors, then your function will probably work on vectors. For example, here's the very first function we wrote:

```
function res = myfunc(x)
    s = sin(x);
    c = cos(x);
    res = abs(s) + abs(c);
end
```

And lo! It turns out to work on vectors because the MATLAB basic functions we used in `myfunc`, `sin`, `cos`, `abs` all work with vectors:

```
>> Y = myfunc(1:3)
Y = 1.3818    1.3254    1.1311
```

Some other functions we wrote don't work on vectors, but they can be patched up with just a little effort. For example, here's `hypotenuse` from Section 5.1:

```
function res = hypotenuse(a, b)
    res = sqrt(a^2 + b^2);
end
```

This doesn't work on vectors because the `^` operator defaults to matrix operations, using linear algebra, which only works on square matrices.

```
>> hypotenuse(1:3, 1:3)
Error using  ^  (line 51)
Incorrect dimensions for raising a matrix to a power.
Check that the matrix is square and the power is a scalar.
To perform element-wise matrix powers, use '.^'.
```

But convert this to use the element-wise array operation by replacing `^` with the element-wise operator (`.^`), it works!

```
>> A = [3,5,8];
>> B = [4,12,15];
>> C = hypotenuse(A, B)

C = 5    13    17
```

The function matches up corresponding elements from the two input vectors, so the elements of `C` are the hypotenuses of the pairs $(3, 4)$, $(5, 12)$, and $(8, 15)$, respectively.

In general, if you write a function using only element-wise operators and functions that work on vectors, the new function will also work on vectors.

1.1.4 Sums and Differences

Another common vector operation is *cumulative sum*, which takes a vector as an input and computes a new vector that contains all the partial sums of the original. In math notation, if V is the original vector, the elements of the cumulative sum, C, are

$$C_i = \sum_{j=1}^{i} V_j$$

In other words, the ith element of C is the sum of the first i elements from V. MATLAB provides a function named `cumsum` that computes cumulative sums:

```
>> X = 1:5

X = 1     2     3     4     5

>> C = cumsum(X)

C = 1     3     6    10    15
```

The inverse operation of `cumsum` is `diff`, which computes the difference between successive elements of the input vector.

```
>> D = diff(C)

D = 2     3     4     5
```

Notice that the output vector is shorter by one than the input vector. As a result, MATLAB's version of `diff` is not exactly the inverse of `cumsum`. If it were, we would expect `cumsum(diff(X))` to be `X`:

```
>> cumsum(diff(X))

ans = 1     2     3     4
```

But it isn't.

Exercise 1.1. Write a function named `mydiff` that computes the inverse of `cumsum` so that `cumsum(mydiff(X))` and `mydiff(cumsum(X))` both return `X`.

1.1.5 Products and Ratios

The multiplicative version of `cumsum` is `cumprod`, which computes the *cumulative product*. In math notation, that's

$$P_i = \prod_{j=1}^{i} V_j$$

In MATLAB, that looks like

```
>> V = 1:5

V = 1     2     3     4     5

>> P = cumprod(V)

P = 1     2     6    24   120
```

MATLAB doesn't provide the multiplicative version of `diff`, which would be called `ratio`, and which would compute the ratio of successive elements of the input vector.

Exercise 1.2. Write a function named `myratio` that computes the inverse of `cumprod`, so that `cumprod(myratio(X))` and `myratio(cumprod(X))` both return `X`.

You can use a loop, or if you want to be clever, you can take advantage of the fact that $e^{\ln a + \ln b} = ab$.

1.2 Computing with Vectors

In this section, we'll look at two common patterns for working with vectors and connect them to the corresponding ideas from mathematics, existential and universal quantification. And you'll learn about logical vectors, which contain the Boolean values 0 and 1.

1.2.1 Existential Quantification

It's often useful to check the elements of a vector to see if there are any that satisfy a condition. For example, you might want to know if there are any positive elements. In mathematical terms, checking whether something exists is called *existential quantification*, and it's denoted with the symbol $\exists$, which is pronounced "there exists." For example,

$$\exists x \text{ in } S : x > 0$$

means, "there exists some element x in the set S such that $x > 0$." In MATLAB, it's natural to express this idea with a logical function, like `exists`, that returns `1` if there is such an element and `0` if there is not.

```
function res = exists(X)
    for i=1:length(X)
        if X(i) > 0
            res = 1;
            return
```

```
        end
    end
    res = 0;
end
```

We haven't seen the `return` statement before ; it's similar to `break` except that it breaks out of the whole function, not just the loop. That behavior is what we want here because as soon as we find a positive element, we know the answer (it exists!) and we can end the function immediately without looking at the rest of the elements.

If we get to the end of the loop, that means we didn't find what we were looking for, so the result is `0`.

1.2.2 Universal Quantification

Another common operation on vectors is to check whether *all* the elements satisfy a condition, which is called *universal quantification*, denoted with the symbol $\forall$ and pronounced "for all." So the expression

$$\forall x \text{ in } S : x > 0$$

means "for all elements x in the set S, $x > 0$."

One way to evaluate this expression in MATLAB is to reduce the problem to existential quantification, that is, to rewrite

$$\forall x \text{ in } S : x > 0$$

to the following:

$$\sim\exists x \text{ in } S : x \leq 0$$

where $\sim\exists$ means "does not exist." In other words, checking that all the elements are positive is the same as checking that there are no elements that are nonpositive.

Exercise 1.3. Write a function named `forall` that takes a vector and returns `1` if all the elements are positive and `0` if there are any non-positive elements.

1.2.3 Logical Vectors

When you apply a logical operator to a vector, the result is a *logical vector*: a vector whose elements are the logical values 1 and 0. Let's look at an example:

```
>> V = -3:3

V = -3    -2    -1     0     1     2     3

>> L = V>0

L =  0     0     0     0     1     1     1
```

In this example, `L` is a logical vector whose elements correspond to the elements of `V`. For each positive element of `V`, the corresponding element of `L` is `1`.

Logical vectors can be used like flags to store the state of a condition. And they are often used with the `find` function, which takes a logical vector and returns a vector that contains the indices of the elements that are "true."

Applying `find` to `L` from the example above yields

```
>> find(L)

ans = 5     6     7
```

which indicates that elements 5, 6, and 7 have the value 1.

If there are no "true" elements, the result is an empty vector.

```
>> find(V>10)

ans = Empty matrix: 1x0
```

This example computes the logical vector and passes it as an argument to `find` without assigning it to an intermediate variable. You can read this version abstractly as "find the indices of elements of `V` that are greater than 10."

We can also use `find` to write `exists` more concisely:

```
function res = exists(X)
    L = find(X>0)
    res = length(L) > 0
end
```

Exercise 1.4. Write a version of `forall` using `find`.

1.3 Debugging in Four Acts

When you're debugging a program, and especially if you're working on a hard bug, there are four things to try:

Reading Examine your code, read it back to yourself, and check that it means what you meant to say.

Running Experiment by making changes and running different versions. Often, if you display the right thing at the right place in the program, the problem becomes obvious, but you might have to invest time building scaffolding.

Ruminating Take some time to think! What kind of error is it: syntax, runtime, or logical? What information can you get from the error messages or from the output of the program? What kind of error could cause the problem you're seeing? What did you change last, before the problem appeared?

Retreating At some point, the best thing to do is back off, undoing recent changes, until you get back to a program that works and that you understand. Then you can start rebuilding.

Beginning programmers sometimes get stuck on one of these activities and forget the others. Each activity comes with its own failure mode. For example, reading your code might help if the problem is a typographical error, but not if the problem is a conceptual misunderstanding. If you don't understand what your program does, you can read it 100 times and never see the error, because the error is in your head.

Running experiments can help, especially if you run small, simple tests. But if you run experiments without thinking or reading your code, you might fall into a pattern I call "random walk programming," which is the process of making random changes until the program does the right thing. Needless to say, random walk programming can take a long time.

The way out is to take more time to think. Debugging is like an experimental science. You should have at least one hypothesis about what the problem is. If there are two or more possibilities, try to think of a test that would eliminate one of them.

Taking a break sometimes helps with the thinking. So does talking. If you explain the problem to someone else (or even yourself), you will sometimes find the answer before you finish asking the question.

But even the best debugging techniques will fail if there are too many errors or if the code you are trying to fix is too big and complicated. Sometimes the best option is to retreat, simplifying the program until you get to something that works, and then rebuild.

Beginning programmers are often reluctant to retreat, because they can't stand to delete a line of code (even if it's wrong). If it makes you feel better, copy your program into another file before you start stripping it down. Then you can paste the pieces back in, a little bit at a time.

To summarize, here's the Eighth Theorem of Debugging:

> Finding a hard bug requires reading, running, ruminating, and sometimes retreating. If you get stuck on one of these activities, try the others.

1.4 Chapter Review

This chapter presents patterns for working with vectors, including existential and universal quantification. We learned how to write functions that take vectors as input variables and return vectors as output variables. And we learned about *logical vectors*, which contain the values 1 and 0 to represent true and false.

Some functions in this chapter are not idiomatic MATLAB; many of them can be done more simply using built-in MATLAB operators and functions, rather than writing them yourself. But these examples demonstrate concepts you will need to know when you work on more complicated problems.

In the next chapter, we will apply the tools we have learned so far to the central goal of this book, modeling physical systems.